

Manufacturing Simulation: Game Specification

Agentic Engineering Arena | Environment #1 of 3 | Replit Build Spec

This document defines the complete game specification for the Manufacturing Simulation environment, the first of three game demos in the Agentic Engineering Arena platform. It covers the grid-based world model, entity types, agent roles, economic system, action spaces, rate limiters, and the shared schema/API contracts that will be reused across all future game environments (Disaster Response and Peer-Agent Swarm). The design draws from the deep-research report's factory environment recommendation and extends it with a profit-driven economic layer, five specialized agent types, and a modern retro visual style.

Design Philosophy: Retro game logic (discrete grid, turn-based ticks, tile-based entities) with a modern visual skin. Think clean vector sprites, smooth animations, and a Roblox-style color palette rather than pixel-art grain. The simulation must be playable as a standalone game while also exposing clean API hooks for the evolutionary algorithm layer and LLM-based agent orchestration.

1. World Model

1.1 Grid Structure

The factory floor is a square grid. The default configuration is 12x12 (144 cells), but the grid size is a configurable parameter to support evolutionary mutation. Each cell is one of the following terrain types:

Cell Type	Walkable?	Description
Floor	Yes	Open factory floor. Agents can walk, items can rest here temporarily.
Wall	No	Structural boundary. Blocks movement and line of sight.
Conveyor	Yes (directional)	Moves items automatically in a fixed direction each tick. Agents can walk on conveyors but items placed on them slide.
Machine Slot	Agents only	A cell occupied by a machine. Agents can interact but cannot pass through when a machine is installed.
Loading Dock (In)	Yes	Raw materials spawn here. Procurement agents purchase goods that appear at these docks.
Shipping Dock (Out)	Yes	Finished products are delivered here for sale. Sales agents finalize transactions.
Storage Zone	Yes	Designated buffer areas for raw materials, intermediates, or finished goods. Limited capacity per cell.

1.2 Coordinate System and Adjacency

Cells are addressed as (row, col) starting from (0,0) at the top-left corner. Adjacency is 4-directional (north, south, east, west) for movement. Diagonal interactions are not allowed for movement but may be allowed for communication range checks (8-directional, Chebyshev distance). The grid wraps at boundaries with walls by default.

1.3 Visibility and Fog of War

Each agent has a visibility radius (default: 3 cells, Chebyshev distance). Agents can only observe entities within their visibility range, except for Management agents who have full map visibility. This creates the asymmetric information structure described in the deep-research report, where floor workers see only their local area while managers see the full picture.

2. Entity Catalog

2.1 Machines

Machines are the core transformation units. Each machine occupies one cell and converts input goods into output goods according to a recipe. Machines have configurable properties that serve as genes in the evolutionary layer.

Machine Type	Input	Output	Base Ticks	Base Cost	Power Draw/Tick
Smelter	Raw Ore (x2)	Metal Ingot (x1)	4	\$500	\$2
Stamping Press	Metal Ingot (x1)	Stamped Part (x1)	3	\$400	\$3
Assembly Station	Stamped Part (x2) + Circuit (x1)	Subassembly (x1)	6	\$800	\$4
Quality Control	Subassembly (x1)	Inspected Unit (x1) or Reject	2	\$300	\$1
Packaging Unit	Inspected Unit (x1)	Finished Product (x1)	2	\$200	\$1
Circuit Fabricator	Raw Silicon (x1)	Circuit (x1)	5	\$600	\$3

Machine States: Each machine cycles through: Idle (waiting for input), Loading (agent placing inputs), Processing (countdown timer running), Output Ready (product waiting for pickup), Broken (random failure, requires Engineering agent to repair), and Offline (powered down to save costs).

Speed Settings: Each machine has three speed modes. Low (0.5x speed, 0.5x power draw, lower failure rate), Normal (1x speed, 1x power draw, baseline failure rate), and High (1.5x speed, 2x power draw, higher failure rate). Speed mode is an evolvable parameter and can be changed by Engineering or Management agents.

2.2 Goods (Items)

Items are discrete objects that occupy cells or are carried by agents. Each item has a type, a value, and a weight (affects agent movement speed when carried).

Item Category	Examples	Purchase Cost	Sale Price	Weight
Raw Materials	Raw Ore, Raw Silicon	\$10-\$20 each	N/A (cannot sell)	Heavy (slows agent 50%)
Intermediate Goods	Metal Ingot, Stamped Part, Circuit	N/A (produced)	N/A (no direct sale)	Medium (slows agent 25%)
Subassemblies	Subassembly, Inspected Unit	N/A (produced)	N/A (no direct sale)	Medium (slows agent 25%)
Finished Products	Finished Product	N/A (produced)	\$150-\$300 each	Light (slows agent 10%)
Rejects	Defective Unit	N/A	\$5 scrap value	Light

Note: All dollar values above are starting parameters. The evolutionary engine can mutate costs and prices. The key constraint is that the production chain must be economically viable at baseline: total input cost + operating cost < sale price.

2.3 Recipe DAG (Production Graph)

The production chain forms a directed acyclic graph (DAG). The default recipe for the MVP:

Path A: Raw Ore (x2) → [Smelter] → Metal Ingot → [Stamping Press] → Stamped Part

Path B: Raw Silicon (x1) → [Circuit Fabricator] → Circuit

Merge: Stamped Part (x2) + Circuit (x1) → [Assembly Station] → Subassembly → [Quality Control] → Inspected Unit → [Packaging Unit] → Finished Product

This DAG is stored as a JSON recipe object and is an evolvable parameter. The evolutionary engine can add parallel paths, change input quantities, or insert new machine stages.

3. Agent Roles and Action Spaces

Five agent types operate on the factory floor. Each has a unique set of actions plus a shared set of common actions. Agents are hired at a cost and paid a wage per tick. Agent counts and types are evolvable parameters.

3.1 Dual-Layer Action System (Critical Design)

The action system operates at two abstraction levels. This is essential because cell-by-cell movement (e.g., calling `move_north` eight times to cross the floor) is appropriate for RL-trained policies but catastrophically inefficient for LLM-based agents, where each decision incurs token cost and API latency. Both layers resolve to the same underlying simulation ticks, preserving determinism.

Micro-Actions (Low-Level)

These are the atomic simulation primitives. Scripted agents and RL policies operate at this level. Each micro-action consumes exactly one tick.

Micro-Action	Ticks	Description
<code>move(direction)</code>	1	Move one cell in the given direction (N/S/E/W). Fails if cell is blocked or occupied.
<code>wait()</code>	1	Do nothing. Agent remains in place.
<code>broadcast(message)</code>	0	Send a structured message within communication range (5 cells). Rate-limited: 1 per 3 ticks.
<code>observe()</code>	0	Request current observation snapshot. Automatically provided each tick.

Macro-Actions (High-Level, LLM-Optimized)

These are compound actions that the server decomposes into micro-action sequences using built-in pathfinding (A*). LLM agents issue one macro-action and the server executes the underlying micro-steps automatically, returning the result when the macro-action completes. This reduces an LLM agent's decision points from hundreds per run to dozens, making real-time play feasible.

Macro-Action	Tick Cost	Description
<code>go_to(target_cell)</code>	Path length	Server computes A* path, agent auto-moves. Aborts and returns partial progress if path becomes blocked mid-traversal.
<code>pickup_nearest(item_type)</code>	Path + 1	Finds closest item of given type within visibility, moves to it, picks it up.
<code>deliver_to(target_cell)</code>	Path + 1	Moves to target and drops carried item. Used for machine loading and dock delivery.
<code>work_machine(machine_id)</code>	Path + processing	Moves to machine, loads input (if carrying), starts processing, waits, unloads output. Full cycle.

Macro-actions are interruptible. If a higher-priority event occurs mid-execution (e.g., machine breakdown, new order), the agent receives an interrupt observation and can choose to abort the current macro-action. The agent is billed only for ticks actually consumed. All macro-actions decompose deterministically into micro-actions, so replay logs and EA fitness comparisons remain valid regardless of which layer the agent used.

Implementation note: The macro-action layer is a convenience wrapper, not a separate system. Internally it queues micro-actions onto the agent's action buffer. Scripted baseline agents can use either layer.

3.2 Procurement Agent

Role: Purchases raw materials from external suppliers and brings them to the production floor. Acts as the supply chain interface.

Hire Cost: \$200 | **Wage:** \$3/tick | **Carry Capacity:** 2 items | **Move Speed:** 1 cell/tick

Unique Action	Ticks	Precondition	Effect
purchase(item_type, qty)	2	Must be on a Loading Dock cell. Budget must cover cost.	Spawns purchased items on the Loading Dock. Deducts cost from budget.
stockpile(item)	1	Must be on a Storage Zone. Carrying the item.	Places item into storage. Storage cell has max capacity of 6 items.
pickup(item)	1	Item on current cell or adjacent cell. Carry capacity not full.	Picks up item. Movement slowed based on item weight.
drop(item)	1	Carrying the item.	Places item on current cell.
check_prices()	0	None	Returns current market prices for raw materials (prices fluctuate slightly each tick cycle).

3.3 Operations / Floor Worker Agent

Role: The primary production worker. Loads/unloads machines, carries items between stations, and performs the core assembly work.

Hire Cost: \$150 | **Wage:** \$2/tick | **Carry Capacity:** 1 item | **Move Speed:** 1 cell/tick

Unique Action	Ticks	Precondition	Effect
load_machine(machine_id)	1	Adjacent to machine. Carrying correct input item(s).	Places item into machine input queue. Machine starts processing if all inputs met.
unload_machine(machine_id)	1	Adjacent to machine. Machine output_ready = true. Carry capacity not full.	Picks up output item from machine.
pickup(item)	1	Item on current or adjacent cell.	Picks up item.
drop(item)	1	Carrying item.	Drops item on current cell.
start_machine(machine_id)	1	Adjacent to machine. Machine is idle with sufficient inputs loaded.	Begins the processing countdown.
request_help(task_desc)	0	None	Broadcasts a help request to nearby agents. Rate-limited.

3.4 Engineering / Maintenance Agent

Role: Repairs broken machines, adjusts machine speed settings, installs and removes equipment, and handles facility modifications.

Hire Cost: \$350 | **Wage:** \$5/tick | **Carry Capacity:** 0 items (tools only) | **Move Speed:** 1 cell/tick

Unique Action	Ticks	Precondition	Effect
repair(machine_id)	3-5	Adjacent to broken machine.	Restores machine to Idle state. Duration depends on machine complexity.
set_speed(machine_id, mode)	1	Adjacent to machine.	Changes machine speed to Low/Normal/High. Takes effect next processing cycle.
install_machine(type, cell)	5	On the target cell. Cell must be a Machine Slot. Budget must cover machine cost.	Places a new machine. Deducts cost from budget.
remove_machine(machine_id)	3	Adjacent to machine. Machine must be Idle or Offline.	Removes machine from grid. Refunds 30% of original cost.
move_machine(machine_id, dest)	8	Adjacent to machine. Destination must be empty Machine Slot.	Relocates machine. Machine is offline during move.
diagnose(machine_id)	1	Adjacent to machine.	Returns machine health %, failure probability, and recommended maintenance schedule.

3.5 Sales Agent

Role: Receives incoming orders, delivers finished products to the Shipping Dock, and finalizes sales transactions. Also manages demand forecasting.

Hire Cost: \$250 | **Wage:** \$4/tick | **Carry Capacity:** 2 items | **Move Speed:** 1 cell/tick

Unique Action	Ticks	Precondition	Effect
sell(item)	2	Must be on a Shipping Dock cell. Carrying a Finished Product.	Sells the product. Adds sale price to budget. Order fulfilled.
check_orders()	0	None	Returns list of pending customer orders with deadlines and bonus/penalty info.
pickup(item)	1	Finished Product on current or adjacent cell.	Picks up finished product.
drop(item)	1	Carrying item.	Drops item on current cell.
negotiate_price(order_id)	1	Active order exists.	Attempts to increase sale price by 10-20% with a success probability based on demand.
forecast_demand()	0	None (cooldown: 20 ticks)	Returns predicted order volume for next 50 ticks.

3.6 Management Agent

Role: Has full map visibility but cannot carry items or operate machines directly. Issues directives, reassigns workers, and makes strategic decisions about hiring/firing and budget allocation.

Hire Cost: \$500 | **Wage:** \$8/tick | **Carry Capacity:** 0 items | **Move Speed:** 1 cell/tick | **Visibility:** Full map

Unique Action	Ticks	Precondition	Effect
hire(agent_type)	3	Budget covers hire cost.	Spawns a new agent of the specified type at a Loading Dock.
fire(agent_id)	1	Agent must exist and not be mid-action.	Removes agent from simulation. No refund.
assign_task(agent_id, task)	0	Agent must be in communication range or same team.	Sends a prioritized task directive to the specified agent. Agent receives it as a high-priority observation.
set_budget_allocation(category, amount)	0	Total allocations must not exceed current budget.	Allocates budget to categories: procurement, equipment, wages, or reserve.
view_financials()	0	None	Returns full P&L: revenue, costs by category, current budget, profit/loss.
approve_purchase(request_id)	0	Pending purchase request exists.	Approves or denies a large purchase (above threshold).

4. Economic System

4.1 Starting Conditions

Each simulation run begins with a finite budget and a basic factory layout. The starting configuration is an evolvable parameter, but the default is:

Parameter	Default Value	Evolvable?
Starting Budget	\$10,000	Yes
Grid Size	12x12	Yes (range: 8x8 to 20x20)
Starting Machines	1 Smelter, 1 Stamping Press, 1 Assembly Station, 1 QC, 1 Packaging Unit, 1 Circuit Fab	Yes
Starting Agents	1 Procurement, 2 Operations, 1 Engineering, 1 Sales, 1 Management	Yes
Raw Material Prices	Ore: \$10, Silicon: \$15	Yes (fluctuation range)
Finished Product Base Price	\$200	Yes
Simulation Length	500 ticks	Yes (range: 200-2000)
Order Arrival Rate	1 order every 15 ticks (avg)	Yes

Economic sanity check: With 6 starting agents (total wage ~\$24/tick) and 6 machines (total power ~\$14/tick when all running), the baseline burn rate is ~\$38/tick, or \$19,000 over 500 ticks. At 1 order per 15 ticks with a \$200 base price, maximum possible revenue is ~33 orders x \$200 = \$6,600. This means the simulation is intentionally cash-constrained: profitability requires either high machine utilization (minimizing idle power), lean staffing, or successfully negotiating higher prices on rush orders. The \$10,000 starting budget provides enough runway (~260 ticks at full burn) to reach first revenue, but the factory must become self-sustaining. This tension is the core strategic challenge.

4.2 Revenue Streams

Revenue comes from selling finished products at the Shipping Dock. Base sale price is \$200, but it varies based on order urgency (rush orders pay 1.5x), negotiation success (Sales agent skill), and market demand cycles (sinusoidal fluctuation over the simulation). Late deliveries incur a penalty of \$20 per tick past the deadline. Rejected/defective units can be sold as scrap for \$5.

4.3 Cost Structure

Costs fall into five categories, all tracked in the P&L:

- **Raw Materials:** Paid at time of purchase by Procurement agent.
- **Equipment:** One-time install cost + ongoing power draw per tick while machine is in Idle or Processing state. Machines in Offline state draw zero power. This makes power management a strategic decision.
- **Labor:** Wage per tick per agent. Active agents pay full wage. Agents placed in 'idle standby' by Management pay 50% wage but cannot act until reactivated (1 tick delay). This gives Management a meaningful cost-control lever.
- **Modifications:** Moving machines (\$50 flat + lost production time), removing machines (no refund beyond 30% salvage).
- **Opportunity Cost:** Unfulfilled orders result in lost revenue and a reputation penalty that reduces future order frequency.

4.4 Objective Function

The primary objective is **cumulative profit** over the simulation run:

$$\text{Profit} = \text{Total Revenue} - (\text{Material Costs} + \text{Equipment Costs} + \text{Labor Costs} + \text{Modification Costs} + \text{Penalties})$$

For longer-horizon simulations, we use **profitability rate** (profit per tick, averaged over the final 25% of the run) to reward sustainable configurations over one-time gains. The evolutionary fitness function can weight these differently:

$$\text{Fitness} = w1 * \text{Cumulative Profit} + w2 * \text{Profitability Rate} + w3 * \text{Throughput} - w4 * \text{Avg Latency} - w5 * \text{Idle Agent Ratio}$$

Default weights: $w1=0.4$, $w2=0.3$, $w3=0.15$, $w4=0.1$, $w5=0.05$. These are tunable by the user or evolved by a meta-EA.

5. Rate Limiters and Game Constraints

Rate limiters create the bottlenecks and coordination challenges that make the simulation interesting. They prevent trivial solutions and force meaningful agent coordination.

Constraint	Value	Purpose
Agent Movement	1 cell per tick (base). Carrying heavy items: 1 cell per 2 ticks.	Creates spatial reasoning. Agents must plan routes.
Cell Occupancy	Max 1 agent per cell. Items can share cells (up to 4).	Forces pathfinding and collision avoidance.
Machine Queue	Max 3 items in input queue per machine.	Creates upstream bottlenecks. Overproduction has consequences.
Communication Rate	1 broadcast per 3 ticks per agent.	Prevents communication spam. Forces prioritization of messages.
Purchase Cooldown	Procurement can only buy every 5 ticks.	Prevents infinite material dumping. Budget pacing.
Repair Duration	3-5 ticks depending on machine type. Engineer is locked during repair.	Engineers are scarce resources. Downtime matters.
Machine Failure Rate	1-5% chance per processing cycle (varies by speed setting).	Introduces stochastic disruptions. High speed = high risk.
Order Deadlines	Each order has a deadline (20-80 ticks from arrival).	Creates time pressure. Late penalties accumulate.
Storage Capacity	6 items per Storage Zone cell.	Prevents infinite stockpiling. Inventory management matters.
Hire Delay	New agents appear 3 ticks after hiring.	Prevents instant workforce scaling. Planning required.

5.2 Conflict Resolution Rules

When multiple agents attempt conflicting actions on the same tick, the simulation resolves them deterministically using the following priority stack:

- **Priority 1 - Cell Occupancy:** If two agents attempt to move into the same empty cell, the agent with the lower agent_id wins (deterministic tiebreak). The losing agent's move fails and they remain in place, consuming their tick.

- **Priority 2 - Item Contention:** If two agents attempt to pick up the same item, the agent already on the item's cell wins over an agent on an adjacent cell. If both are equidistant, lower agent_id wins.
- **Priority 3 - Machine Access:** Only one agent can interact with a machine per tick. If two agents attempt to load/unload/start the same machine, the agent with the role-appropriate specialization wins (e.g., Operations beats Procurement for load_machine). Same-role ties use agent_id.
- **Priority 4 - Budget Contention:** If two agents attempt budget-spending actions (purchase, hire, install) that would jointly exceed the remaining budget, they are processed in agent_id order. The second action fails if insufficient funds remain.

Implementation note: All actions are collected per tick, sorted by agent_id, validated against current state, and applied sequentially. The state is committed atomically after all actions resolve. This is functionally equivalent to simultaneous resolution with deterministic tiebreaks.

6. Game UX Principles

6.1 Visual Style

The visual style is **retro logic, modern skin**. The game uses discrete grid-based mechanics (tile movement, turn-based ticks, cell occupancy) but renders with clean vector art, smooth CSS/canvas animations, and a bright, readable color palette. Think of the clarity of a Roblox game applied to a 2D factory sim. Specific guidelines:

- Agents are represented as distinct character sprites with role-specific color coding and icons (hardhat for Operations, wrench for Engineering, briefcase for Sales, clipboard for Procurement, tie for Management).
- Machines are chunky, recognizable icons with animated state indicators: green glow = running, yellow pulse = output ready, red flash = broken, gray = offline.
- Items are small, colored tokens that visually stack when multiple items share a cell.
- Conveyors show directional arrows and animate items sliding along them.
- The grid has subtle gridlines with a clean industrial floor texture, not pixel-art grain.

6.2 Information Display

- **HUD Overlay:** Current budget, tick counter, profit/loss running total, pending orders count.
- **Tooltips:** Hovering over any entity shows its full state (machine health, agent inventory, item type).
- **Mini-map:** A small overview of the full grid in the corner, showing agent positions and hotspots.
- **Alert System:** Visual and audio pings for critical events: machine breakdown, order deadline approaching, budget low, agent idle for 10+ ticks.

6.3 Replay and Debugging

Every simulation run is recorded as a sequence of (tick, actions, state_delta) tuples. The UI supports full replay with play/pause/step-forward/step-back controls. A 'blame mode' overlay highlights which agent or machine caused the most recent throughput drop or cost spike. This supports both human understanding and evolutionary fitness debugging.

6.4 Speed Controls

The simulation runs at configurable speed: 1x (real-time, ~2 ticks/second for observation), 5x, 10x, and Max (headless, no rendering, for EA batch evaluation). The headless mode is critical for the evolutionary loop, which may evaluate hundreds of configurations per generation.

7. Shared API Schema (Cross-Environment Contract)

The following API contract is shared across all three game environments (Manufacturing, Disaster Response, Peer-Agent Swarm). Environment-specific details are injected via configuration, but the interface is uniform. This enables the evolutionary engine and agent orchestration layer to treat all environments as interchangeable modules.

7.1 Core API Endpoints

Endpoint	Input	Output	Description
reset(config)	EnvironmentConfig JSON	InitialState JSON	Initialize or reinitialize the environment with a given genome/config. Returns starting state with all entity positions, budgets, recipes.
step(actions)	Dict of {agent_id: Action}	(next_state, observations, rewards, done, info)	Advance the simulation one tick. Apply all agent actions, resolve conflicts, update state, return per-agent observations.
get_metrics()	None	MetricsSnapshot JSON	Return current aggregated telemetry: throughput, latency, costs, queue lengths, idle times, profit.
get_state()	None	FullState JSON	Return complete current state (for replay logging or EA evaluation).
render()	Optional: render_config	Frame data (canvas commands or serialized visual state)	Return a visual frame for the UI. Skipped in headless mode.
get_action_space(agent_id)	agent_id string	ActionSpace JSON	Return the list of valid actions for a given agent at the current tick, including preconditions.
get_observation(agent_id)	agent_id string	Observation JSON	Return the observation for a specific agent (respecting visibility rules).

7.2 Data Schemas

EnvironmentConfig

The config object encodes the genome for a simulation run. All fields are optional with defaults. For Manufacturing, this includes grid_size, machine_list (types, positions, speeds), agent_list (types, starting positions), recipe_dag, starting_budget, order_arrival_rate, raw_material_prices, and simulation_length. Future environments replace machine_list with environment-specific entities (e.g., road_network for Disaster Response).

State / Observation

State is a JSON object with top-level keys: tick (integer), agents (dict of agent_id to AgentState), machines (dict of machine_id to MachineState), items (list of ItemState), grid (2D array of cell types), budget (float), orders (list of pending Order objects), and metrics (running MetricsSnapshot). Observations are filtered views of State based on the requesting agent's visibility radius and role.

Action

An Action is a JSON object with fields: type (string, e.g. 'move', 'pickup', 'repair'), params (dict of action-specific parameters), and agent_id (string). Invalid actions return a failure status and the agent's tick is consumed (no free retries).

MetricsSnapshot

Contains: throughput (products completed), avg_latency (ticks from raw to finished), total_revenue, total_costs (broken down by category), current_profit, agent_idle_ratio (0.0-1.0), machine_utilization (0.0-1.0 per machine), queue_lengths (per machine), and orders_fulfilled / orders_missed.

7.3 WebSocket Event Protocol

The frontend connects via WebSocket for real-time updates. Events are pushed from the server on each tick:

- **tick_update:** Full state delta (only changed fields) after each step.
- **agent_action:** Per-agent action log with success/failure status.
- **alert:** Critical event notifications (machine failure, budget warning, order deadline).
- **metrics_update:** Updated MetricsSnapshot at configurable intervals (default: every 5 ticks).
- **game_over:** End-of-simulation signal with final metrics and fitness score.

The client can also send commands via WebSocket: pause, resume, set_speed, and manual_action (for human-in-the-loop mode where a user controls one agent).

8. Evolutionary Interface Hooks

The simulation must expose clean hooks for the evolutionary algorithm layer. These hooks allow the EA to mutate configurations, run batch evaluations, and extract fitness scores without touching the game's internal logic.

8.1 Genome Encoding

The genome for the Manufacturing environment is a JSON object with the following mutable fields:

Gene Category	Fields	Mutation Operations
Topology	Machine positions on grid, conveyor layout, wall placement	Swap two machine positions, add/remove conveyor segment, toggle wall cell
Machine Config	Speed settings (Low/Normal/High), machine types present, count of each type	Change speed mode, add/remove a machine, duplicate a machine type
Agent Config	Agent types present, count of each, starting positions	Add/remove agent, change agent type ratio, randomize starting positions
Recipe DAG	Production graph edges, input quantities per recipe step	Add parallel path, change input quantity +/- 1, swap two recipe steps
Economic Params	Starting budget, raw material prices, sale prices, wage rates	Perturb by +/- 10%, swap two price values
Timing Params	Order arrival rate, simulation length, machine failure rates	Perturb by +/- 15%, toggle failure on/off for a machine type

8.2 Batch Evaluation Protocol

The EA runs simulations in headless mode. The protocol is: (1) Call reset(config) with a candidate genome. (2) Loop: call step(agent_actions) until done=true. Agent actions can come from scripted policies, random policies, or LLM agents depending on the evaluation mode. (3) Call get_metrics() to retrieve the final fitness components. (4) Return the fitness score to the EA. Each evaluation must be deterministic given a fixed random seed, enabling reproducible comparisons between genomes.

8.3 Fitness Extraction

After each run, the EA extracts a fitness vector: [cumulative_profit, profitability_rate, throughput, avg_latency, agent_idle_ratio, machine_utilization, orders_fulfilled_ratio]. The EA's selection function maps this vector to a scalar fitness using configurable weights (see Section 4.4). Multi-objective Pareto selection is also supported for exploration of tradeoff surfaces.

9. Implementation Guidance

This section provides directional guidance for the Replit build without being overly prescriptive about implementation details.

9.1 Suggested Architecture

- **Frontend:** React with PixiJS or Phaser for the simulation canvas. React Flow for the DAG visualizer panel (right side of the split-screen). Standard React components for HUD, tooltips, and controls.
- **Backend:** Node.js or Python (FastAPI) running the simulation engine. WebSocket connection for real-time state pushes. REST endpoints for configuration, replay, and batch evaluation.
- **State Management:** The simulation state is authoritative on the server. The client receives state deltas and renders them. No game logic runs on the client.
- **Agent Interface:** Agents connect as WebSocket clients or HTTP polling clients. Each agent receives its observation and returns an action. This abstraction supports both scripted agents (for testing), LLM-based agents (via tool calls), and human players (via UI controls).

9.2 File Structure Suggestion

Organize the codebase to separate concerns cleanly:

- **/engine** - Core simulation logic: grid, entities, tick processing, conflict resolution. This is environment-agnostic where possible.
- **/environments/manufacturing** - Manufacturing-specific configs: recipes, machine types, agent roles. Future environments get sibling directories.
- **/api** - The shared API layer (endpoints, WebSocket handlers, serialization).
- **/agents** - Agent implementations: random, scripted, LLM-based. Each implements the same Action interface.
- **/evolution** - EA loop: genome encoding, mutation operators, fitness extraction, selection.
- **/ui** - React frontend: simulation canvas, DAG panel, HUD, replay controls.

9.3 Key Technical Decisions to Make

The following decisions are left to the implementation team, as they depend on Replit's runtime constraints and the team's preferences:

- Whether to use PixiJS (lighter, more control) or Phaser (more built-in game engine features) for rendering.
- Whether the backend is Node.js (easier WebSocket integration, same language as frontend) or Python (better ML/EA library ecosystem).
- How to handle concurrent agent actions within a single tick (random order, priority order, or simultaneous resolution with conflict rules).
- Whether to use SQLite, in-memory state, or a lightweight DB for replay log persistence.
- The specific LLM integration pattern for agent decision-making (direct API calls, LangGraph orchestration, or a custom tool-calling wrapper).

9.4 Testing Strategy

Start with the simplest possible scenario and layer complexity incrementally:

- **Phase 1:** Get items flowing. Spawn raw materials at a dock, have one Operations agent carry them to one Smelter, produce an ingot. Verify the state machine works.
- **Phase 2:** Complete the production chain. Wire up the full recipe DAG with all six machine types. Run one product through end-to-end.
- **Phase 3:** Add economics. Track budget, costs, and revenue. Verify profit calculation is correct.
- **Phase 4:** Add multiple agents. Test collision avoidance, communication, and task assignment.
- **Phase 5:** Hook up the EA. Run two configurations headless and compare fitness scores.
- **Phase 6:** Polish the UI. Add animations, tooltips, replay controls, and the DAG panel.

10. Pre-Mortem: Failure Modes and Design Amendments

We conducted a pre-mortem analysis to identify the most likely failure modes before implementation begins. Each issue below includes the root cause, the consequence if unaddressed, and the design amendment we have incorporated into this spec.

10.1 LLM Agent Latency Kills Real-Time Play

Root Cause: An LLM agent making one API call per tick at ~2 seconds latency means a 500-tick simulation takes 16+ minutes per agent. With 6 agents, serial execution is over 90 minutes. The EA needs hundreds of runs per generation.

Amendment: We introduced the dual-layer action system (Section 3.1). LLM agents issue macro-actions (e.g., `go_to`, `work_machine`) that the server decomposes into micro-action sequences. This reduces LLM decision points from ~500 to ~30-50 per run, bringing a single simulation under 2 minutes. For EA batch evaluation, scripted or random policies at the micro-action level run in milliseconds headlessly.

10.2 The Genome Is Too Large for Course-Timeline EA

Root Cause: Six gene categories with mixed continuous and discrete parameters create a combinatorial search space that even a well-tuned EA cannot explore meaningfully in 5-10 generations with 9-15 candidates per generation (the Taguchi L9 approach from the PRD).

Amendment: We define three genome complexity tiers. For the MVP and VC demo, use Tier 1 only. Tiers 2-3 are unlocked as the platform matures.

Tier	Mutable Parameters	Search Space Size	Target Use
Tier 1 (MVP)	Agent counts by type (5 integers), machine speed settings (6 categorical), order arrival rate (1 float)	~12 dimensions	VC demo. Enough to show meaningful optimization. Fits L9 array.
Tier 2	Tier 1 + machine positions on grid, conveyor layout	~30 dimensions	Post-MVP. Demonstrates spatial optimization.
Tier 3	Tier 2 + recipe DAG edges, economic params, agent capabilities	~60+ dimensions	Research mode. Requires larger populations and more generations.

10.3 No Built-In Proof That Evolution Beats Baseline

Root Cause: The entire value proposition rests on showing that evolved configurations outperform hand-designed ones. But the v1 spec had no baseline policies, no A/B comparison infrastructure, and no way to visually demonstrate the improvement to a VC in a 10-minute pitch.

Amendment: We added Section 11 (Baseline Comparison Framework) defining three built-in baseline agent policies and a side-by-side comparison mode. The demo scenario (Section 12) includes a pre-configured 'bad' baseline and a known-good evolved configuration so the improvement is visible on first run.

10.4 Economic Model Can Bankrupt Before First Sale

Root Cause: The v1 spec used a \$5,000 starting budget with ~\$24/tick labor costs and ~\$14/tick machine costs. At full burn, the factory runs out of money in ~130 ticks. But the first finished product requires at minimum: purchasing raw materials (~\$35 for one unit's inputs), processing through 4-5 machines (~20+ ticks), and delivering to the dock. Even with perfect coordination, first revenue might not arrive until tick 40-60. With imperfect agents, it could be tick 100+.

Amendment: Starting budget increased to \$10,000 (Section 4.1). Additionally, machines in Offline state draw zero power, and agents can be temporarily idled at half wage. This gives management agents meaningful cost-control levers rather than making bankruptcy a function of how fast the agents stumble through early ticks.

10.5 Tick-Decision Coupling Assumes Homogeneous Agent Speed

Root Cause: If every agent must submit one action per tick, and ticks advance synchronously, then the slowest agent (the LLM making an API call) gates the entire simulation. In a mixed environment with scripted and LLM agents, this means scripted agents wait idly while the LLM thinks.

Amendment: The simulation supports two execution modes. **Synchronous mode** (default for replay and UI) advances one tick at a time, waiting for all agents. **Async-buffered mode** (for live play with mixed agent types) allows agents to submit actions at their own pace. The server buffers actions and advances ticks on a fixed wall-clock interval (e.g., every 500ms). Agents that miss the window execute `wait()` by default. This is configured per-run, not per-agent.

10.6 No Validation That Evolved Configs Are Physically Playable

Root Cause: An EA that mutates grid layout could produce configurations where machines are unreachable (walled off), docks are disconnected from the production floor, or there is no valid path from input to output. The simulation would run but produce zero throughput, wasting an evaluation slot.

Amendment: Every genome must pass a **connectivity validator** before evaluation. The validator checks: (1) All machines are reachable from at least one Loading Dock via walkable cells. (2) At least one Shipping Dock is reachable from every machine. (3) No agent spawn point is trapped. Invalid genomes are rejected and re-mutated. This is a simple BFS check that runs in milliseconds and prevents the EA from wasting compute on dead configurations.

11. Baseline Comparison Framework

To demonstrate the value of evolutionary optimization, the platform ships with three built-in agent policy tiers. The VC demo runs all three on the same environment configuration and displays the results side-by-side.

11.1 Policy Tier Definitions

Policy Tier	Agent Logic	Expected Behavior	Purpose
Random	Each tick, select a uniformly random valid action from the action space.	Chaotic. Agents wander aimlessly, occasionally stumble into productive actions. Very low throughput, high idle time, frequent budget exhaustion.	Floor baseline. Proves the game is not trivially solvable.
Scripted (Greedy)	Hard-coded priority rules: (1) If carrying item, deliver to nearest appropriate machine. (2) If not carrying, pick up nearest available item. (3) If no items visible, move toward nearest machine with <code>output_ready</code> . (4) Else wander.	Functional but inefficient. Agents complete the production chain but create bottlenecks at popular machines, ignore communication, and make no strategic decisions about hiring, speed settings, or purchasing timing.	Realistic 'hand-coded' baseline. Represents the current state of the art in manually scripted agent workflows.
Evolved (EA-Optimized)	Same scripted logic but with EA-tuned parameters: agent count ratios, machine speed settings, spatial layout, and order pacing.	Noticeably better throughput, fewer bottlenecks, higher profit. The improvement is visible in the DAG visualizer (shorter queues, fewer red edges) and the P&L (higher cumulative profit).	Demonstrates the platform's value. The delta between Scripted and Evolved is the product's core selling point.

11.2 Comparison Mode UI

The comparison mode displays two simulation runs side-by-side in a split view. The left panel runs the Scripted baseline; the right panel runs the Evolved configuration. Both use the same random seed and order sequence. A shared timeline at the bottom synchronizes playback. Key metrics (throughput, profit, idle ratio) are displayed as overlaid line charts updating in real-time, making the performance delta immediately visible. At the end of the run, a summary card highlights the percentage improvement across each metric.

11.3 Minimum Viable Delta Target

For the VC demo to be compelling, the Evolved configuration should achieve at least a 25-40% improvement in cumulative profit over the Scripted baseline. Based on our economic model, a Tier 1 EA optimizing agent counts and machine speeds should be able to achieve this by discovering that: (a) one additional Operations agent eliminates the carrier bottleneck between Smelter and Assembly, (b) running the Smelter at High speed and the Packaging Unit at Low speed balances the pipeline, and (c) reducing Procurement to 1 agent from 2 avoids over-purchasing raw materials. These are non-obvious optimizations that a human might miss but an EA can find in 5-10 generations.

12. Pre-Built Demo Scenario: 'First Factory'

The platform ships with one fully configured scenario that works out of the box. This scenario is the default when a user (or VC) first opens the application. It is tuned to complete a full run in under 3 minutes at 5x speed and to produce a visually obvious difference between baseline and evolved configurations.

12.1 Scenario Configuration

Parameter	Value
Grid Size	10x10 (smaller than default for faster runs and clearer visuals)
Layout	Loading Dock at (0,0) and (0,1). Shipping Dock at (9,9). Machines arranged in a rough L-shape from top-left to bottom-right. 2 Storage Zone cells near the center. Walls forming a corridor that creates a natural bottleneck at (5,5).
Machines	1 Smelter at (2,2), 1 Circuit Fab at (2,6), 1 Stamping Press at (4,4), 1 Assembly Station at (6,4), 1 QC at (7,6), 1 Packaging at (8,8). All at Normal speed.
Agents (Baseline)	1 Procurement, 1 Operations, 1 Engineering, 1 Sales, 1 Management. (Understaffed by design to create a visible bottleneck.)
Starting Budget	\$8,000
Simulation Length	300 ticks
Order Rate	1 order every 12 ticks
Random Seed	42 (fixed for reproducibility)

12.2 Expected Baseline vs. Evolved Outcome

With the baseline Scripted policy on this scenario, the single Operations agent becomes the critical bottleneck. It spends most of its time walking between machines because the L-shaped layout forces long traversals. The Smelter and Circuit Fab produce intermediates faster than the lone worker can deliver them, causing queue buildup. Orders start timing out around tick 150.

The Tier 1 EA, after 5-8 generations with a population of 9 (L9 array), should discover a configuration that adds 1-2 Operations agents and sets the Assembly Station to High speed (the actual bottleneck machine). This simple change should increase throughput by 30%+ and turn the P&L from negative to positive, creating a clear 'before and after' visual for the VC pitch.

12.3 What the VC Sees

The demo flow is: (1) Open the app, which loads the First Factory scenario. (2) Click 'Run Baseline' to watch the Scripted agents struggle for 60 seconds at 5x speed. Key visual: growing red queue indicators at the Assembly Station, order deadline warnings flashing in the HUD. (3) Click 'Run Evolution' to kick off a 5-generation EA search (runs headlessly in ~30 seconds using scripted agents at Max speed). (4) Click 'Compare' to see the side-by-side split view with the best evolved configuration. Key visual: shorter queues, faster product flow, profit line going up instead of down. (5) Inspect the DAG panel to see which parameters the EA changed and why.

This specification is a living document. Values, action sets, and parameters are designed to be adjusted through playtesting and evolutionary tuning. The critical invariants are: (1) the API contract (Section 7), which must remain stable across all three game environments; (2) the dual-layer action system (Section 3.1), which must be implemented before any LLM agent integration; and (3) the connectivity validator (Section 10.6), which must gate all EA evaluations. Everything else can be tuned.